



~/fx-sales-intervention – zsh

```
# author:  Aditya Singh
```

```
# context:  FX Sales Workstation · 5d · 4 agents · 1 prototype
```

```
$ ./ship.sh --hold-the-wheel
```

```
>>> The War Rig Drives Itself
```

We're using the smallest part of this

Across the whole build, AI tends to sit in a few familiar spots — and only at the shallow end. The surface is much larger. much larger.

Discovery & requirements	
Specification & scoping	
Architecture decisions	
Writing code	• in use today · autocomplete
Boilerplate & scaffolding	• in use today · generation
Orchestrating the work	
Tests & verification	• in use today · sample tests only
Documentation	• in use today · summaries
Review & sign-off	

An agent can sit at every step

■ enhance what's in use

■ explore new ground

The spots you already use AI can go much deeper — and the rest of the surface is open.

Discovery & requirements	agent drafts & synthesises · you decide
Specification & scoping	agent turns notes into a spec · you set scope
Architecture decisions	agent proposes options + ADRs · you choose
Writing code	from autocomplete → full features you review
Boilerplate & scaffolding	from snippets → whole modules you direct
Orchestrating the work	agents run in parallel · you conduct
Tests & verification	from sample tests → the contract for correct
Documentation	from summaries → a living, verified wiki
Review & sign-off	agent lints & flags · you approve

What we'll cover

§1

The Workflow

How spec-anchored agentic development differs from vibe coding. The five rules. The audit trail.

§2

Discovery

The structured interview that produced eleven documents before a single line of code was written.

§3

Orchestration

Four agents, five sprints, thirty-four tickets.

§4

Tests

The data-* contract, the self-healing debug loop, and why tests were the spine not the scaffolding.

§1

The Workflow

Vibe coding vs. spec-anchored agentic development.

From problem statement to shippable workstation in five days — *auditable* end to end.

Three modes of LLM-assisted development

Vibe coding

- One conversation, prompt-driven
- No persistent spec or contract
- Output reviewed only at the end
- Drift invisible until shipped

Structured agentic

- Multi-step plans
- Light spec or backlog
- Tests written after code
- Some traceability

Spec-anchored

- Spec pack precedes code (~1,500 lines)
- TDD intensity per ticket (● / ● / ●)
- Cross-team escalation discipline
- Phase-gate lint catches drift

The current build sat in column three. *Every rule below comes from that column.*

Five rules that made it auditable

- 1 Spec pack first.** 11 documents, ~1,500 lines. Every ticket cites sections.
- 2 TDD intensity labelled per ticket.**  Strict ·  Alongside ·  Acceptance. 23 of 34 strict-or-alongside.
- 3 Write boundaries.** Dev/SDET → src/tests. PM → docs. BA → wiki.
- 4 Bors-style merge queue.** No direct pushes to main. Bisections on failure.
- 5 Phase-gate lint sweep.** BA runs an 10-category drift check at every phase close.

docs/BACKLOG.md · FXSW-010

FXSW-010 · dealMachine cross-model

Effort: L · TDD:  Strict

Docs: 03-trade-state-model §3, §4

TDD tests (write first):

- Pickup → SI Initial→PickUpSent→PickedUp
- Hold from PickedUp → HoldSent → Initial
- Quote → QuoteSent → Quoted; RFS Executable
- Withdraw, Reject, ClientReject paths
- after: 5000 → Removed (terminal states)
- Terminal states reject subsequent events

Done when: 10 tests pass; coverage ≥ 90%.

The audit layer - a living wiki

A separate agent kept a living wiki — the synthesis layer that kept everything traceable.

wiki/

| overview.md

what it is, current state

| glossary.md

FX + SI terminology

| log.md

every ingest + lint sweep

| features/

blotter, ticket, AI panel

| components/

state machines, feeds, engine

| data-models/

deal, client-profile, tick

| decisions/

ADR-0001 ... ADR-0010

| scenarios/

the 5 E2E walkthroughs

Independent

A separate agent owns the wiki — and never touches the code it documents.

Traceable

Every page cites the spec doc or ticket it came from.

Verified

Lint greps the real source each phase. Drift is surfaced, never overwritten.

Every line of code is traceable

Commit → Ticket → Spec section → Wiki/ADR → Lint sweep



wiki/log.md · excerpt

```
## [2026-05-26] lint | Post-Phase-4 lint pass (MANDATORY focus: engine.ts pip-deltas ↔  
## wiki/components/suggestion-engine.md). Findings:  
## - Pip-delta drift check: CLEAN. 13/13 values match – 4 tier baselines (1.5/2/3/4),  
## 3 notional bands (+2.5/+1.5/+0.5), 3 market deltas, behaviour deltas, floor.  
## - LINT-501 (escalated): docs/BACKLOG.md shows FXSW-034 as ☐ but workflow  
## shipped at commit 0762c4e. Write boundary forbids fix. Routed to build.
```

When code, spec, and wiki drift, the lint catches it. *When the BA can't fix it (write boundary), it escalates.*

How did Claude know what to build?

Before answering — three numbers from the build.

316

unit tests · 0 todo

35.9s

for 6 E2E tests

0

new deps after Sprint 1

Next: §2 · **Discovery** — the interview that built the spec.

§2

Discovery

Before any code — three sessions, eleven documents.

Claude interviewed me for five hours. The result was a *spec pack* the team could build from.

Three sessions over two days

Session 1

Tue AM · 90 min

Problem framing

- Problem statement, six goals
- Three personas, fifteen user stories
- Scope, non-goals, constraints
- Six measurable success criteria

Session 2

Tue PM · 2 hours

Domain modelling

- Two state machines (RFS + SI)
- Cross-model relationship table
- Five demo scenarios with data
- Five-second removal rule

Session 3

Wed AM · 2 hours

UI · stack · tests

- Token palette + component inventory
- Vite, XState + Zustand, AG-Grid
- Suggestion-engine rules + 5 profiles
- TDD intensity per ticket

5h 15min interview → 11 docs · ~1,500 lines · 10 ADRs · 34 tickets across 5 sprints

A real exchange · Session 1

The moment the credit-breach behaviour was reframed.

...

Aditya

Add: When credit would be breached, I want a decline recommendation rather than a wider price.

Claude

Number 13 is interesting. The AI should not suggest pricing — it should suggest declining. That's the AI as a guardrail, not a gas pedal.

Aditya

That's exactly the framing. The "smart" moment is knowing when not to price. Anyone can widen a margin; the judgement call is recognising the deal shouldn't be done.

Claude

I'll capture that as a design principle — the AI engine has a special case for credit-limit credit-limit breaches, different output shape.

What this produced

PRD

User story #13 locked

ADR-0007

AI = guardrail framing

docs/09 §7

Credit-decline output shape

FXSW-026

Credit-decline panel ticket

engine.ts

Short-circuit branch

CREDIT_BREACH

End-to-end test scenario

The spec pack

Eleven documents the team built from.

docs/

└ 00-glossary.md	62 lines
└ 01-prd.md	108 lines
└ 02-functional-spec.md	204 lines
└ 03-trade-state-model.md	187 lines
└ 04-dummy-feed-spec.md	131 lines
└ 05-ui-ux-spec.md	243 lines
└ 06-tech-architecture.md	198 lines
└ 07-scenario-pack.md	152 lines
└ 08-test-plan.md	121 lines
└ 09-suggestion-engine.md	164 lines
└ BACKLOG.md	34 tickets

10 ADRs locked at sign-off

ADR-0001	Vite over Next.js
ADR-0002	Two state machines (RFS + SI)
ADR-0003	XState + Zustand
ADR-0007	AI as a guardrail, not a gas pedal
ADR-0008	Indigo accent reserved for AI surfaces
ADR-0009	Simulated ack delays (250ms, zeroable)
ADR-0010	Brand-neutral build

Per-ticket TDD intensity assigned

 Strict	Tests first, run red, then code	11 tickets
 Alongside	Tests written with code	17 tickets
 Acceptance	Gherkin scenario IS the test	6 tickets

Next: §3 · **Sprint orchestration** — what the team did with this.

§3

Orchestration

Four agents. Five sprints. Thirty-four tickets.

What happened when the `spec` became code.



What's agent orchestration?

Multiple specialised LLM agents coordinating through structured channels — not one model doing everything.

The principle

Each agent has a fixed **role** (PM, Dev, SDET, BA), a **write boundary** (which folders it can touch), and a **toolkit**.

They coordinate through structured channels — escalations with severity, mail with subject lines, a merge queue with verification gates — not freeform conversation.

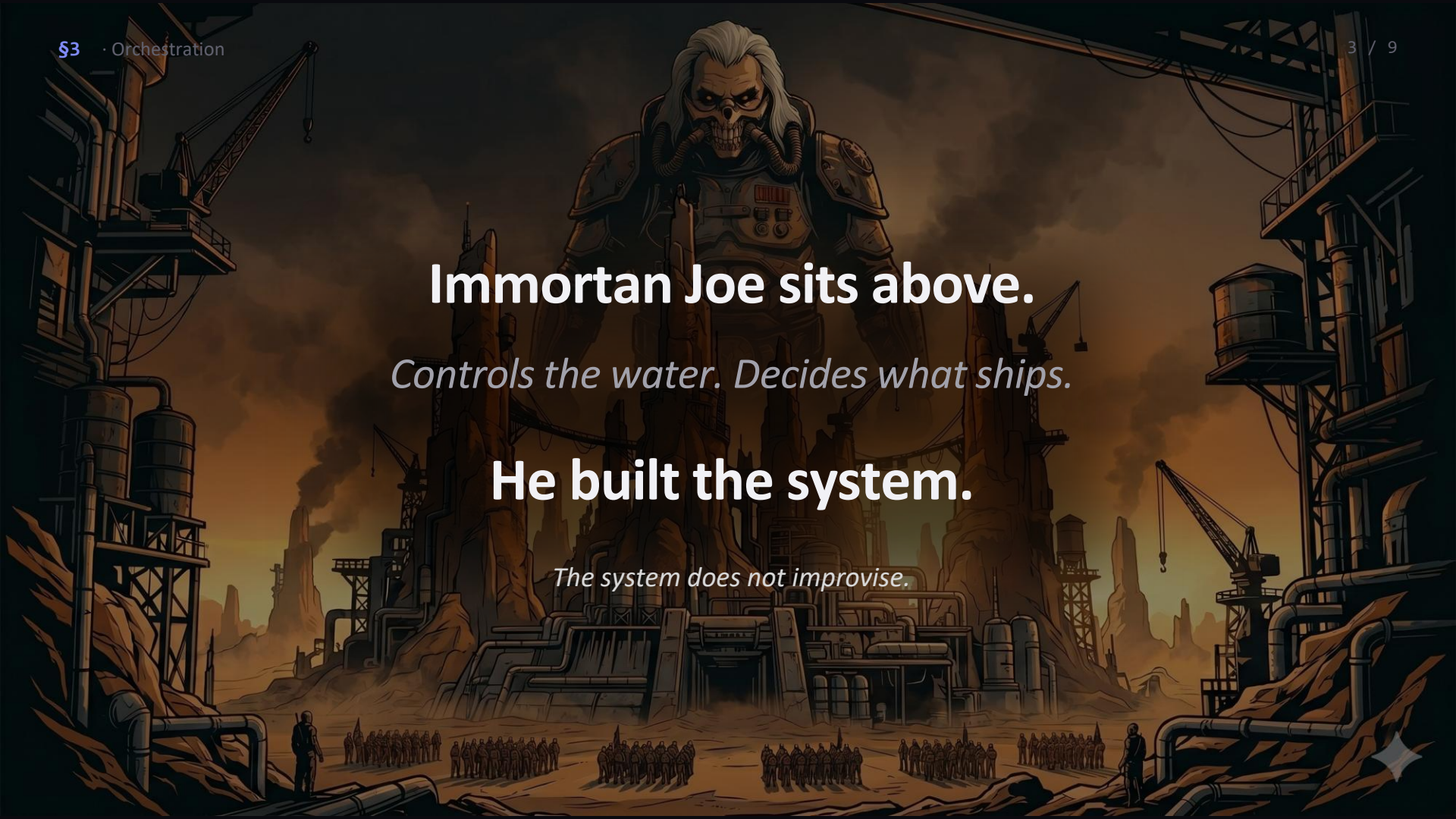
The orchestration layer is what turns *"one model doing everything"* into *"a team where each agent does what its role demands."*

Gas Town

Steve Yegge's framework. What it gives you:

Mayor	Planner agent · attaches at session start
Polecat	Ephemeral worker · one per ticket
Witness	Per-rig test gate · catches drift
Refinery	Bors-style merge queue (one at a time)
Scribe	Long-lived doc-owner polecat
Bead	= ticket; convoy = sprint
Escalate	Severity-routed problem channel
gt CLI	The interface agents use

This build used Gas Town with a four-role cast. *The cast is on the next slide.*

A dark, industrial landscape with a giant skeletal figure in the center. The figure is wearing a suit with a skull for a head and has long white hair. It stands amidst a complex of pipes, cranes, and industrial structures. In the foreground, there are several groups of small, uniformed figures standing in formation. The sky is dark and smoky.

Immortan Joe sits above.

Controls the water. Decides what ships.

He built the system.

The system does not improvise.

The Warboys (Team)

Four agents, one overseer. Two vocabularies for the same cast.

Product (Immortan Joe) Scope decisions · tiebreakers · mid-flight amendments · live validation



PM

Mayor

Sprint planning · ticket coordination · escalation · phase summaries summaries

Dev

Polecat-N (ephemeral)

Implementation per spec · TDD execution · technical debate

SDET

Witness

Test slate authorship · gate enforcement · drift detection

BA

Polecat-scribe (long-lived)

Wiki authorship · ADR drafting · phase-gate lint sweeps

Five sprints in one day

Each row: tickets · one moment per sprint that was worth a decision.

1**Scaffold + live URL**

FXSW-001 → 006, 034

*FXSW-034 pulled forward. Live URL on Day 1.***2****Feed + state**

FXSW-007 → 013

*ESP terminal invariant break. Mid-flight responsive amendment.***3****Ticket panels**

FXSW-014 → 021

*Seed-42 rounding asymmetry. HoldButton inline now, lift later.***4****AI Margin Suggestion**

FXSW-022 → 027

*Halcyon neutral prior. @vitest/coverage-v8 skipped.***5****Notifications + ship**

FXSW-028 → 033

Set.size $O(1)$ chime. Final-sweep lint: LINT-501 open.

One moment, told two ways · Halcyon neutral prior

Sprint 4 · FXSW-022 · code review before merge.

Sprint log · Sprint 4 · FXSW-022

Decision · FXSW-022 — Halcyon neutral prior

Dev encoded the five client profiles per the spec table. Halcyon's `recent30dAcceptanceRate` was listed in the spec as `"—"` (no `"—"` (no history — new client). Dev's first draft encoded the missing missing value as `0`

SDET intercepted at code review before merge. The suggestion engine engine had a rule: `acceptance < 0.4` deducted 0.5 pips. A new client with client with no quotes would have been penalised on day one.

"No data" ≠ "data showing zero." A `Beta(1,1)` prior on a Bernoulli with Bernoulli with no observations is 0.5. Dev applied the neutral prior with prior with an inline comment so the next reader doesn't "fix" it back to 0.

Build log · Convoy 4 · 08:43

```
[polecat-022] first draft: Halcyon recent30d = 0
[polecat-022] tests green · gt done?
[witness.fxsw] holding · code-review before merge
```

```
to: polecat-022 (Dev)
from: witness.fxsw (SDET)
subj: Halcyon's 0 will trip engine penalty
```

Halcyon is a NEW client. No quotes yet, no history. Encoding 0 means we penalise them on day 1. Semantically wrong: `"no data" ≠ "data showing zero."` `Beta(1,1)` prior on a Bernoulli with no observations is 0.5. Use neutral prior.

— witness.fxsw

```
[polecat-022] applying · Halcyon = 0.5
[polecat-022] gt done · 59fff0e → main
```

Same event. Same commit (59fff0e). *Different vocabulary for different audiences.*

Another moment · mid-flight responsive amendment

Sprint 2 · Day 2 evening · Product opened the live URL on a phone.

Build log · Convoy 2 · 16:39

[overseer] viewing live URL on phone

to: mayor.fxsw (PM)
from: overseer (Product)
subj: live URL on mobile is unusable

Header cut off, blotter rows wider than viewport. PRD §4 scopes mobile out — but I'd like to demo on my phone at a coffee.

[mayor.fxsw] two options:

- (a) amend PRD, card-stacked redesign
- (b) horizontal-scroll containment ~1h

[overseer] Path b. Go.

```
$ gt convoy add convoy.fxsw.c2 fxsw-resp  
✓ slung fxsw-resp → polecat-resp  
[witness.fxsw] playwright at 3 viewports  
  · 390×844 ✓ · 768×1024 ✓  
[polecat-resp] reconciling 4 doc files
```

Sprint log · Sprint 2

Decision · Mid-sprint amendment

Late on Day 2, Product opened the live URL on mobile. The PRD PRD explicitly scoped responsive out — but Product wanted to demo to demo on a phone.

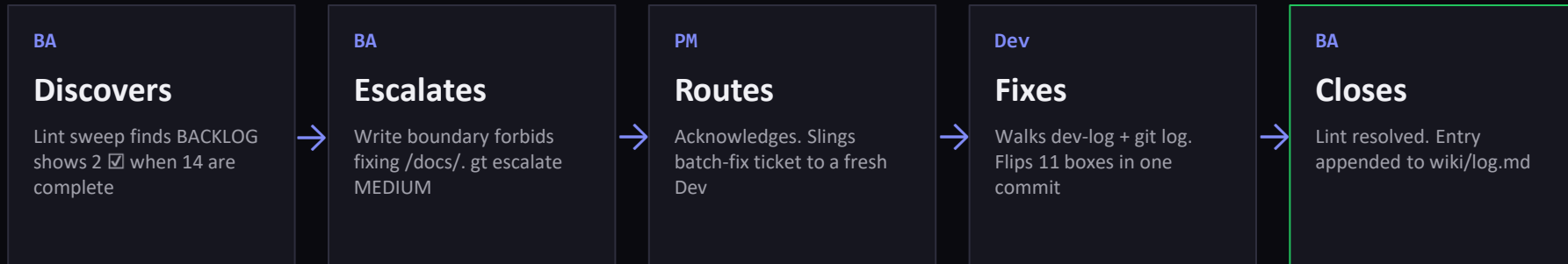
PM proposed two paths: (1) amend the PRD and execute a card-stacked card-stacked redesign — Sprint-5-shaped work. (2) Single layout with layout with horizontal-scroll containment, accept "usable, not pretty." pretty."

Decision: option 2. FXSW-resp added to the active sprint. Four Four documents reconciled. PM directed Dev to document the the amendment in [dev-log.md](#) as the *precedent for future in-flight flight changes*.

Out-of-scope concern, same escalation channel. *The amendment became part of the operating model.*

Problems get named, routed, fixed, closed

Worked example · LINT-002 · BACKLOG status drift



Build log · the actual sequence

```
[polecat-scribe-01] LINT-002 (escalated): BACKLOG status column 11 unflipped vs git log
$ gt escalate -s MEDIUM "LINT-002 · Wiki agent write boundary forbids editing docs/."
✓ escalation gt-esc-005 created · routed to mayor.fxsw
[mayor.fxsw] gt-esc-005 acknowledged · slinging batch-fix bead to polecat-bk
[polecat-bk] flipping fxsw-001 through fxsw-022 to [X] in one commit
[polecat-bk] gt done · 58895d4 → main
[mayor.fxsw] gt-esc-005 closed
```

Five days, one prototype

The final ledger.

Throughput

33/34

tickets delivered

1

mid-flight
amendment

39

polecats spawned

38

merges to main

Discipline

0

silent overwrites

0

new deps after
Sprint 1

2

spec deviations
inline

10

ADRs at sign-off

Resilience

7

escalations total

6

closed in-session

1

open (LINT-501)

0

lost to silent failure

Next: §4 · **Tests — the thread that held it all together.**

§4

Testing

The spine, not a deliverable.

What held the build together across five days, four agents, *and one self-healing debug loop.*

Three layers, three questions

Each layer answers exactly one question. Each ticket cited which layer to use.

Vitest

Unit

Does this pure function compute the right number?

Scope

lib/ · machines/ · suggestion/

Floor

≥90% lines · 100% on engine

Example

PRNG seed-42 first EURUSD tick = 1.1715

Vitest + RTL

Component

Does this component render the right thing?

Scope

PricingPanel · SuggestionPanel

Floor

≥80% on critical surfaces

Example

Apply triggers data-margin-glow on input

Playwright

E2E

Does the user's journey work?

Scope

5 named scenarios + smoke

Floor

All 5 scenarios passing

Example

OFF_HOURS → Quoted → TradeConfirmed in 8.2s

316 unit · 6 E2E in 35.9s. *Every ticket cited which layers to write before the code.*

The data-* contract

Assert on attributes, never on text or color. Text changes; canonical state names don't.

What Playwright sees · a live row

```
<button
  data-testid="deal-row"
  data-deal-id="d_a4f2"
  data-si-state="Quoted"
  data-rfs-state="Executable"
  data-display-status="STREAMING"
  data-dealable="false"
  data-removing="false"
>
  <span>Globex Industries</span>
  <span>USDJPY · SELL</span>
  <span>5,000,000</span>
</button>
```

What the E2E asserts · OFF_HOURS scenario

```
// after Send Stream + ack
const row = page.locator(
  '[data-deal-id="d_a4f2"]'
);

await expect(row).toHaveAttribute(
  'data-si-state', 'Quoted'
);

await expect(row).toHaveAttribute(
  'data-display-status', 'STREAMING'
);

// 1.5s scripted CLIENT_ACCEPT
await expect(row).toHaveAttribute(
  'data-si-state', 'TradeConfirmed'
);
```

Text can change. Colors can change. *State names from the spec don't.*

The self-healing loop

Claude Desktop · Browser MCP · running scenarios, observing the DOM, fixing the source, until green.



Run

```
pnpm test:e2e size-limit-margin-tune
```

Observe

```
<div data-testid="suggestion-pips">4<span>...</span>
```

Diagnose

```
toHaveText reads innerText → "4pips" not "4"
```

Fix

```
Scope testid to value-only child
```

Re-run

```
9.2s · PASS · captured to test-patterns.md §8
```

The loop is autonomous. *The spec is the leash.*

Receipts

Test discipline, locked at ship.

316

unit tests · 0 todo

35.9s

for 6 E2E scenarios

100%

branch coverage on engine
engine

11

patterns captured for reuse

0

scenarios broken at ship

Spec at the top. **Tests** at the bottom. **Code** in between.

Five days. Thirty-four tickets. One prototype.

What you can reach for now

Avenues to explore in your own workflow.

Dev

typing code → shaping it

- Spec-first: write the ticket + tests, tests, agent drafts the code
- Offload boilerplate & scaffolding entirely
- Have the agent draft ADRs as you make the call
- Generate a state machine from a spec, then review it

QA / SDET

finding bugs → defining correct

- Turn user scenarios into machine-checkable contracts
- Let the agent write E2E from named named flows
- Utilize a self-healing debug loop against the contract
- Define “correct” up front, before the code exists

Dev leads

doing the work → directing it

- Run agents in parallel lanes across a whole feature
- Use agents for monitoring and traceability
- Stand up a separate wiki agent for traceability
- Make drift checks + manage the trust layer

*Stop doing the tasks. **Start directing the work.***

What engineering can do as a whole

Beyond any one role — how the team changes shape when agents carry the work.

Adopt orchestration workflows

Run several agents in defined lanes across a feature. You conduct the work instead of typing all of it.

Hand off the tasks, keep the meat

Delegate scaffolding, boilerplate, and busywork to agents directly. Spend your attention on the hard decisions.

An agent as your personal BA

Document everything as you go — a living wiki, decision logs, traceability — without stopping to write it yourself.

Spec + verification as the deliverable

Treat the spec pack and the trust layer — tests, drift checks, escalation — as the unit of work, not an afterthought.

It's not just engineering

Where the same agents plug into the roles that feed and follow the build.

Product

shaping the build before it starts

- Spin up working prototypes for review — before engineering touches it
- Turn interviews into structured specs the team builds from
- Pressure-test scope and edge cases up front
- Keep the spec pack live as decisions change

Support

closing the loop after it ships

- Capture reported issues into structured tickets
- Run level 0/1 triage and diagnosis with an agent agent
- Pull resolution steps grounded in the actual codebase
- Surface recurring issues back into the spec

The SI machine, documented

Initial	awaiting pickup
PickUpSent	pick up in flight · ~250ms
PickedUp	ticket active, prices streaming
QuoteSent	quote in flight · ~250ms
Quoted	quote live with client
WithdrawSent	withdraw in flight · ~250ms
HoldSent	release in flight · ~250ms
RejectSent	reject in flight · ~250ms
TraderRejected	terminal
ClientRejected	terminal
TradeConfirmed	terminal
Removed	cleanup · triggers archival

The *Sent pattern

Five *Sent states model in-flight actions awaiting backend ack. Each transitions via after: { ackDelay: 'NextState' } — 250ms in production, zero-able in tests.

ADR-0009 · not skippable — they model real latency

Why a Removed state?

XState v5 doesn't allow after transitions on final states. states. Removed is a regular state reached 5 seconds after seconds after any terminal — so the store can observe observe and archive.

Test contract

data-si-state="Quoted" · the primary Playwright assertion attribute

Structured knowledge capture

5-pass rule chain

1. Tier baseline

platinum 1.5 · gold 2 · standard 3 · new 4

2. Notional size

>20M +2.5 · 10-20M +1.5 · 5-10M +0.5

3. Market context

vol >1.5 +1 · thin +1.5 · EM pair +0.5

4. Rejection reason

OFF_HOURS +1.5 · SIZE_LIMIT +0.5

5. Client behaviour

flight_risk -1 · VIP volume -0.5

floor: $\max(1, \text{round}(\text{base} + \sum \text{deltas}))$

Invariant test

$\text{sum}(\text{factor deltas}) + \text{tierBase} === \text{suggestedPips}$ — an explicit algebraic invariant across sampled inputs.

Credit short-circuit

CREDIT_LIMIT bails before the chain. Returns kind:'credit-decline', no suggestedPips, no Apply button. Guardrail documented at rule level.

E2E locked to exact values

Northwind 12M EURUSD SIZE_LIMIT = 4 pips, High confidence. The Playwright E2E asserts on these exact numbers.

Decision traceability

Pre-build decision

AG-Grid Community

De-facto trading-blotter grid



FXSW-012 — superseded

Plain flex-row table

AG-Grid 31: no native per-row data-* API

The Playwright contract runs on **data-deal-id, data-si-state, data-rfs-state, data-display-status** — per-row. No native AG-Grid API for that. Grid was in package.json; it never shipped.

Mark superseded — don't rewrite

ADR-0004 survives with status: superseded and a Superseded-by link. The original reasoning (trading-blotter aesthetic, pre-built renderers) stays in the record. Any future reader wondering why AG-Grid is still in package.json finds the full decision trail.

Continuous verification

Vendor neutrality	✓ clean
State machine drift	✓ siMachine 12 states match src
Dependency versions	✓ package.json exact match
Scenario data	✓ 5/5 match definitions.ts
Client profiles	✓ clean
Pip-delta drift (13/13)	✓ CLEAN – engine.ts verified
data-testid contract	✓ clean
LINT-501 (backlog stale)	escalated → build agent

4 tier baselines · 3 notional bands · 3 market deltas · 2 reason
deltas · 3 behaviour deltas

LINT-501

BACKLOG.md showed FXSW-034 as ☐ open. The deploy had already shipped at commit 0762c4e. Wiki caught it — couldn't fix it — escalated.

Lint greps the source

Pip values in wiki vs engine.ts. State names vs siMachine.ts. Scenario data vs definitions.ts. Checked every phase.

Write boundaries hold

Fixing the backlog required the build agent. The wiki agent can't write to docs/ — so it escalated instead of guessing.